

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)

BÁO CÁO HOMEWORK 2

CHESS AI SỬ DỤNG ALPHA-BETA PRUNING VÀ MCTS

GIẢNG VIÊN: TS. Nguyễn Quốc Minh

NHÓM LỚP: Nhóm 02 - L01

SINH VIÊN:

1. Trần Lê Gia Thoại - 2313323
2. Trần Văn Mạnh - 2212003
3. Nguyễn Đức Thịnh - 2313284
4. Lâm Hoàng Vũ - 2313953
5. Lê Thành Bách - 2210182
6. Trần Trung Hiếu - 2211015
7. Nguyễn Hoàng Danh - 2210456

Tóm tắt (Abstract)

Đề tài này trình bày việc xây dựng một hệ thống chơi cờ vua có tích hợp trí tuệ nhân tạo trong khuôn khổ bài tập lớn môn học. Mục tiêu chính của hệ thống là mô phỏng một ván cờ hoàn chỉnh, xử lý đúng luật chơi cơ bản, đồng thời cho phép so sánh hai hướng tiếp cận tiêu biểu trong AI cho bài toán trò chơi đối kháng là Alpha-Beta Pruning và Monte Carlo Tree Search (MCTS).

Về mặt triển khai, hệ thống được xây dựng bằng ngôn ngữ Python, sử dụng thư viện `python-chess` để biểu diễn trạng thái bàn cờ và quản lý luật chơi, cùng với `pygame` để xây dựng giao diện tương tác. Thuật toán Alpha-Beta được hiện thực theo hướng tối ưu hóa tìm kiếm với các thành phần hỗ trợ như sắp xếp nước đi, tìm kiếm tĩnh và bảng chuyển vị nhằm giảm số lượng trạng thái cần duyệt. Trong khi đó, MCTS được triển khai theo quy trình lựa chọn, mở rộng, mô phỏng và lan truyền ngược, qua đó khai thác đặc trưng thống kê để lựa chọn nước đi.

Kết quả của đề tài không chỉ cho thấy khả năng xây dựng một ứng dụng cờ vua hoạt động được, mà còn góp phần làm rõ sự khác biệt giữa hai chiến lược ra quyết định trong AI: một bên dựa trên tìm kiếm có định hướng và hàm lượng giá, một bên dựa trên mô phỏng và ước lượng xác suất. Từ đó, hệ thống đóng vai trò như một mô hình học tập trực quan, phù hợp cho việc nghiên cứu, quan sát và đánh giá các thuật toán trí tuệ nhân tạo trong môi trường cờ vua.

Mục lục

Danh sách thành viên & Phân công công việc	6
1 Giới thiệu vấn đề (Introduction)	7
2 Động lực thực hiện (Motivations)	8
2.1 Mục tiêu nghiên cứu của phần triển khai	8
2.2 Lý do chọn hai thuật toán	9
2.3 Phạm vi và kỳ vọng	9
3 Nghiên cứu liên quan (Related Work & Literature Review)	9
3.1 Kỹ nguyên tìm kiếm tất định: Minimax và Alpha-Beta	9
3.2 Kỹ nguyên xác suất: MCTS và học sâu	10
3.3 Nghiên cứu đối chiếu và xu hướng lai	10
4 Cơ sở lý thuyết (Background & Theoretical Foundations)	10
4.1 Thuật toán Minimax và Cây trò chơi (Game Tree)	10
4.2 Kỹ thuật Cắt tỉa Alpha-Beta (Alpha-Beta Pruning)	11
4.3 Tìm kiếm cây Monte Carlo (MCTS)	12
4.4 Hàm đánh giá (Evaluation Function)	13
5 Phương án đề xuất (Methodologies)	14
5.1 Kiến trúc hệ thống và giao diện người dùng (GUI)	14
5.1.1 Tổng quan kiến trúc hệ thống	14
5.1.2 Luồng hoạt động chính (Game Loop)	16
5.1.3 Thiết kế giao diện người dùng	18
5.1.4 Minh họa mã nguồn giao diện	19
5.1.5 Đánh giá thiết kế và lựa chọn công nghệ	20
5.2 Triển khai thuật toán AI	20
5.2.1 Giải thuật Alpha-Beta	20
5.2.2 Giải thuật MCTS (Monte Carlo Tree Search)	28

6	Kết quả & Phân tích (Results & Discussion)	32
6.1	Kịch bản thử nghiệm	32
6.2	Đánh giá và So sánh hiệu năng	33
7	Thách thức & Hạn chế (Challenges & Limitations)	34
7.1	Hạn chế về thuật toán (Algorithmic Limitations)	34
7.2	Hạn chế về hệ thống và triển khai (System & Implementation Limitations)	35
8	Kết luận & Hướng phát triển trong tương lai (Conclusion and Future Works)	35
8.1	Kết luận	35
8.2	Hướng phát triển	36
	Tài liệu tham khảo (References)	36

Danh mục hình ảnh

1	Cấu trúc thư mục của hệ thống	15
2	Luồng xử lý tổng thể của hệ thống	16
3	Luồng hoạt động của game loop	16
4	Giao diện chính của trò chơi	18
5	Giao diện menu lựa chọn thuật toán	19
6	Biểu đồ so sánh tỷ lệ thắng giữa Alpha-Beta và MCTS	34



Danh mục bảng biểu

1	Bảng phân công công việc	6
2	So sánh thời gian suy nghĩ trung bình cho một nước đi	33

Danh sách thành viên & Phân công công việc

STT	Họ và tên	MSSV	Nhiệm vụ được giao	% Hoàn thành
1	Lê Thành Bách	2210182	Tổng hợp báo cáo, Title page, Abstract, Table of Contents, Introduction	100%
2	Trần Văn Mạnh	2212003	Nghiên cứu: Phân tích Motivations và Related work / Literature review	100%
3	Lâm Hoàng Vũ	2313953	Lý thuyết: Cây trò chơi, Minimax, Alpha-Beta, MCTS và Hàm đánh giá	100%
4	Trần Trung Hiếu	2211015	Hệ thống & GUI: Game loop, quản lý python-chess và pygame	100%
5	Nguyễn Đức Thịnh	2313284	Thuật toán cốt lõi: Giải thích chi tiết code Alpha-Beta và MCTS	100%
6	Nguyễn Hoàng Danh	2210456	Thử nghiệm: Chạy test, thu thập số liệu, vẽ biểu đồ so sánh hiệu năng	100%
7	Trần Lê Gia Thoại	2313323	Tổng kết: Chỉ ra hạn chế, kết luận, hướng phát triển và References	100%

Bảng 1: Bảng phân công công việc

1 Giới thiệu vấn đề (Introduction)

Cờ vua là một trong những bài toán kinh điển của trí tuệ nhân tạo nhờ đặc trưng đối kháng rõ ràng, không gian trạng thái lớn và yêu cầu ra quyết định tuần tự trong điều kiện luật chơi chặt chẽ. Mỗi nước đi không chỉ tác động đến trạng thái hiện tại mà còn ảnh hưởng trực tiếp đến nhiều diễn biến tiếp theo của ván cờ. Chính vì vậy, cờ vua từ lâu đã được xem là môi trường phù hợp để nghiên cứu các kỹ thuật tìm kiếm trên cây trò chơi, xây dựng hàm đánh giá và kiểm chứng hiệu quả của các chiến lược lựa chọn hành động trong AI.

Trong phạm vi bài tập lớn này, nhóm lựa chọn xây dựng một hệ thống chơi cờ vua có giao diện trực quan, cho phép người dùng tương tác trực tiếp với chương trình và quan sát cách máy đưa ra quyết định. Trọng tâm của đề tài không nằm ở việc xây dựng một chess engine có sức mạnh thi đấu cao như các hệ thống thương mại hoặc nghiên cứu chuyên sâu, mà tập trung vào mục tiêu học thuật: hiện thực hóa, tích hợp và so sánh hai phương pháp tiêu biểu thường gặp trong lĩnh vực trí tuệ nhân tạo cho trò chơi, gồm Alpha-Beta Pruning và Monte Carlo Tree Search (MCTS). Đây là hai hướng tiếp cận có bản chất khác nhau, qua đó giúp làm nổi bật nhiều khía cạnh quan trọng của bài toán ra quyết định trong môi trường đối kháng.

Cụ thể, Alpha-Beta Pruning đại diện cho nhóm phương pháp tìm kiếm có định hướng dựa trên cây trò chơi. Thuật toán này phát triển từ Minimax, khai thác giả định rằng cả hai người chơi đều hành động tối ưu, đồng thời sử dụng các ngưỡng cắt tĩa để loại bỏ những nhánh không cần thiết. Khi được kết hợp với các kỹ thuật như sắp xếp nước đi, tìm kiếm tĩnh và bảng chuyển vị, Alpha-Beta có thể cải thiện đáng kể hiệu quả tính toán trong điều kiện tài nguyên hữu hạn. Ngược lại, MCTS đại diện cho cách tiếp cận dựa trên mô phỏng và thống kê, trong đó việc lựa chọn nước đi được dẫn dắt bởi quá trình cân bằng giữa khai thác những nhánh tiềm năng và khám phá những nhánh chưa được khảo sát đầy đủ. Việc đặt hai thuật toán này trong cùng một hệ thống giúp tạo ra cơ sở trực quan để quan sát sự khác biệt về cơ chế hoạt động, chất lượng nước đi và chi phí xử lý.

Một mục tiêu quan trọng khác của đề tài là gắn kết phần lý thuyết với hiện thực hệ thống. Thay vì chỉ dừng ở mức mô tả khái niệm, nhóm triển khai các thuật toán trên một chương trình có thể vận hành thực tế, trong đó trạng thái bàn cờ được quản lý bằng `python-chess`, giao diện hiển thị và tương tác được xây dựng bằng `pygame`, còn luồng chơi được tổ chức theo mô hình người chơi đấu với AI. Cách tiếp cận này giúp đề tài không chỉ có giá trị về mặt giải thích thuật toán mà còn phản ánh được những vấn đề thường gặp khi đưa một phương pháp AI vào ứng dụng, chẳng hạn như giới hạn độ sâu tìm kiếm, kiểm soát thời gian phản hồi, tổ chức mã nguồn và phối hợp giữa phần xử lý logic với giao diện người dùng.

Bên cạnh đó, bài toán cờ vua cũng cho phép nhóm đánh giá một cách tương đối rõ ràng hiệu quả của từng phương pháp. Với Alpha-Beta, chất lượng quyết định phụ thuộc nhiều vào hàm đánh giá và khả năng cắt tỉa hiệu quả trên cây trò chơi. Với MCTS, độ ổn định của kết quả chịu ảnh hưởng lớn bởi số lần mô phỏng và chất lượng chính sách rollout. Thông qua việc triển khai đồng thời cả hai phương pháp trong cùng một chương trình, đề tài tạo điều kiện để phân tích ưu điểm, hạn chế và tính phù hợp của từng thuật toán trong bối cảnh tài nguyên tính toán còn giới hạn của một bài tập lớn ở bậc đại học.

Từ các định hướng trên, đề tài hướng đến ba mục tiêu chính. Thứ nhất, xây dựng một hệ thống chơi cờ vua có thể hoạt động hoàn chỉnh, có giao diện tương tác cơ bản và xử lý được tiến trình của một ván cờ. Thứ hai, hiện thực hóa hai thuật toán AI là Alpha-Beta Pruning và MCTS trên cùng nền tảng để phục vụ việc nghiên cứu và so sánh. Thứ ba, từ quá trình xây dựng và thử nghiệm, rút ra nhận xét về đặc điểm hành vi, hiệu năng và những hạn chế thực tế của từng phương pháp khi áp dụng vào bài toán cờ vua.

Nhìn chung, ý nghĩa của đề tài không chỉ nằm ở sản phẩm phần mềm hoàn chỉnh ở mức cơ bản, mà còn ở giá trị học thuật và thực hành mà quá trình triển khai mang lại. Đề tài giúp củng cố kiến thức về cây trò chơi, heuristic evaluation, cắt tỉa, mô phỏng xác suất và tổ chức một hệ thống AI có giao diện trực quan. Đồng thời, đây cũng là nền tảng phù hợp để mở rộng sang các hướng phát triển cao hơn trong tương lai như cải thiện hàm đánh giá, tối ưu hiệu năng, tăng cường thử nghiệm hoặc tích hợp thêm các phương pháp học máy hiện đại.

2 Động lực thực hiện (Motivations)

Bài toán xây dựng AI chơi cờ vua là một bài thử tiêu biểu cho các hệ ra quyết định tuần tự trong không gian trạng thái lớn. Với hệ số phân nhánh cao và giới hạn thời gian suy luận trên mỗi nước đi, nhóm cần lựa chọn chiến lược tìm kiếm vừa đủ nhanh, vừa đủ chính xác để có thể triển khai trong ứng dụng thực tế.

Động lực của đề tài xuất phát từ nhu cầu trả lời một câu hỏi thực nghiệm rõ ràng: trong cùng điều kiện cài đặt, **Alpha-Beta Pruning** và **Monte Carlo Tree Search (MCTS)** khác nhau như thế nào về tốc độ ra quyết định, độ ổn định chiến thuật và chất lượng nước đi.

2.1 Mục tiêu nghiên cứu của phần triển khai

- **Mục tiêu 1 – Đối chiếu công bằng:** Thiết lập cùng môi trường chạy, cùng tài nguyên tính toán và cùng giao diện để so sánh hai thuật toán.
- **Mục tiêu 2 – Đo lường định lượng:** Thu thập các chỉ số chính gồm thời gian

suy nghĩ trung bình, kết quả đối đầu và mức độ ổn định theo nhiều trạng thái ván cờ.

- **Mục tiêu 3 – Liên hệ thực tiễn:** Đánh giá thuật toán nào phù hợp hơn cho hệ thống có ràng buộc thời gian phản hồi trong ứng dụng của nhóm.

2.2 Lý do chọn hai thuật toán

- **Alpha-Beta** đại diện cho hướng tìm kiếm tất định, mạnh về chiến thuật cục bộ và hiệu quả khi có cắt tỉa tốt.
- **MCTS** đại diện cho hướng tìm kiếm xác suất, linh hoạt trong khám phá và phù hợp với đánh giá dài hạn.

Việc chọn cặp thuật toán này giúp phép so sánh có ý nghĩa học thuật lẫn kỹ thuật, vì chúng thể hiện hai triết lý tối ưu khác nhau nhưng đều đang được dùng rộng rãi trong các engine hiện đại.

2.3 Phạm vi và kỳ vọng

Phần *Motivation* chỉ xác định bài toán, mục tiêu và tiêu chí đánh giá. Cơ sở lịch sử học thuật và các nghiên cứu nền tảng được trình bày chi tiết ở phần *Related Work & Literature Review*; phần *Results & Discussion* sẽ kiểm chứng lại các giả thuyết bằng số liệu thực nghiệm.

Từ đó, nhóm kỳ vọng rút ra được quy tắc lựa chọn thuật toán theo bối cảnh sử dụng, đồng thời làm nền cho các hướng mở rộng lai (hybrid) trong các phiên bản tiếp theo.

3 Nghiên cứu liên quan (Related Work & Literature Review)

Phần này tổng hợp các công trình nền tảng và xu hướng nghiên cứu chính liên quan trực tiếp đến hai hướng tìm kiếm mà báo cáo sử dụng: *Alpha-Beta* và *MCTS*. Mục tiêu là xác định bối cảnh học thuật mà phần thực nghiệm của nhóm sẽ kiểm chứng.

3.1 Kỹ nguyên tìm kiếm tất định: Minimax và Alpha-Beta

Shannon (1950) đặt nền móng cho việc mô hình hóa cờ vua bằng cây trạng thái và hàm đánh giá; Turing (1951/1953) bổ sung tư duy đánh giá ở trạng thái ổn định (tiền đề của *quiescence search*).

Knuth và Moore (1975) là cột mốc toán học quan trọng khi phân tích hiệu quả của Alpha-Beta và chỉ ra khả năng giảm đáng kể số nút cần duyệt nếu sắp xếp nước đi tốt. Nhờ đó, nhiều kỹ thuật tối ưu như *move ordering*, *transposition tables*, và *null-move pruning* trở thành thành phần cốt lõi của các engine truyền thống.

Deep Blue (Campbell et al., 2002) cho thấy đỉnh cao của hướng tiếp cận này: kết hợp tìm kiếm sâu với tri thức chuyên gia và phần cứng mạnh để đạt hiệu năng chiến thuật rất cao.

3.2 Kỹ nguyên xác suất: MCTS và học sâu

Kocsis và Szepesvári (2006) giới thiệu UCT, giúp MCTS cân bằng giữa khai thác và khám phá theo cơ sở thống kê. Đây là bước ngoặt giúp tìm kiếm Monte Carlo vận hành hiệu quả hơn trong các không gian trạng thái lớn.

AlphaZero (Silver et al., 2018) tiếp tục mở rộng hệ hình bằng cách kết hợp MCTS với mạng nơ-ron sâu cho hai đầu ra *policy* và *value*. Cách kết hợp này giảm phụ thuộc vào luật heuristic thủ công và cải thiện khả năng đánh giá vị trí dài hạn.

3.3 Nghiên cứu đối chiếu và xu hướng lai

Các nghiên cứu đối chiếu cho thấy hai hướng tiếp cận có ưu/nhược điểm bổ sung:

- Alpha-Beta thường mạnh ở chiến thuật ngắn hạn và phát hiện biến bất buộc.
- MCTS thường mạnh ở phân bổ tài nguyên tìm kiếm bất đối xứng và đánh giá dài hạn.

Baier và Winands (2018) đề xuất nhiều biến thể MCTS-Minimax lai, cho thấy việc đưa tín hiệu chiến thuật của Minimax vào pipeline MCTS có thể cải thiện độ ổn định. Ở chiều ngược lại, dòng Stockfish NNUE (từ 2020) tích hợp mạng nơ-ron gọn nhẹ vào hệ Alpha-Beta, minh họa xu hướng hội tụ giữa tìm kiếm cổ điển và học máy.

4 Cơ sở lý thuyết (Background & Theoretical Foundations)

4.1 Thuật toán Minimax và Cây trò chơi (Game Tree)

Lý thuyết cốt lõi:

Cờ vua được phân loại vào nhóm *Zero-sum game* (Trò chơi đối kháng có tổng bằng 0), nghĩa là lợi thế của người này chính là bất lợi của người kia. Trong môi trường này, thuật toán xử lý trò chơi thông qua **Cây không gian trạng thái (Game Tree)**. Nút gốc (Root node) đại diện cho trạng thái hiện tại của bàn cờ, trong khi các nhánh đại diện cho các nước đi hợp lệ (legal moves). Quá trình kết thúc tại các Nút lá (Leaf nodes) khi đạt đến trạng thái kết thúc trò chơi hoặc giới hạn độ sâu.

Thuật toán Minimax mô phỏng một quá trình luân phiên tối ưu giữa hai bên: Người chơi hệ Max (thường là quân Trắng) cố gắng tối đa hóa điểm số, trong khi người chơi hệ Min (quân Đen) cố gắng tối thiểu hóa nó. Đánh giá cuối cùng cho các nhánh được tính ngược từ lá lên gốc (Back-up values).

Liên hệ cài đặt trong hệ thống:

Trong hệ thống của chúng ta, cấu trúc tìm kiếm trên cây được xây dựng dựa trên vòng lặp duyệt qua tất cả các nước đi hợp lệ tính từ vị trí gốc bằng phương thức `board.legal_moves` của thư viện `python-chess`. Mặc dù hệ thống không cài đặt Minimax thuần túy, nhưng nền tảng đệ quy và duyệt cây của Minimax hoàn toàn được tái hiện dưới danh nghĩa của biến thể tối ưu hơn là Alpha-Beta. Tham số `is_max` được sử dụng để phân loại và hoán đổi lượt đi của Trắng hoặc Đen qua mỗi lần đệ quy, kết hợp ăn ý với các thao tác đi thử và hoàn tác trạng thái bàn cờ qua hàm `push(move)` và `pop()`.

4.2 Kỹ thuật Cắt tỉa Alpha-Beta (Alpha-Beta Pruning)

Lý thuyết cốt lõi:

Khuyết điểm lớn nhất của Minimax là sự bùng nổ tổ hợp, số lượng trạng thái cần duyệt tăng theo cấp số mũ dựa trên độ sâu cây. Thuật toán Alpha-Beta ra đời nhằm "cắt tỉa" (prune) các nhánh không tiềm năng và không làm thay đổi kết quả cuối cùng theo tiên đoán. Quá trình này duy trì hai biến giới hạn:

- α : Giá trị lớn nhất mà hàm *MAX* chắc chắn nhận được tính đến thời điểm hiện tại.
- β : Giá trị nhỏ nhất mà hàm *MIN* chắc chắn nhận được.

Nếu bất kỳ lúc nào xuất hiện tình huống $\beta \leq \alpha$, thuật toán có thể dừng việc duyệt các nhánh con còn lại tại nút hiện tại (thực hiện cắt tỉa không khoan nhượng).

Liên hệ cài đặt trong hệ thống:

Tại file `ai/alphabeta.py`, luồng xử lý đệ quy áp dụng quy tắc cắt tỉa này một cách trực tiếp. Ngoài ra, để thao tác cắt tỉa có thể diễn ra *sớm nhất và với số lượng lớn nhất*, hệ thống đã kết hợp thêm ba phương pháp tối ưu cực kỳ quan trọng:

1. **Move Ordering (Sắp xếp nước đi):** Đánh giá ưu tiên tạm thời bằng hàm `order_moves`. Các nước đi *ăn quân* (thường +1000 điểm chuẩn và tính toán bù trừ giá trị quân bài mục tiêu) và *phong cấp* được ưu tiên duyệt nghiệm trước tiên.
2. **Quiescence Search (Tìm kiếm tĩnh):** Hạn chế hiện tượng "Hiệu ứng chân trời" (Horizon Effect) khiến máy tính đưa ra nước cờ hy sinh. Hàm `quiescence_search` buộc máy tính tiếp tục đánh giá sâu hơn ở thời điểm trao đổi quân (không tuân theo giới hạn độ sâu ban đầu) nhằm chắc chắn đánh giá kết quả tại trạng thái tĩnh.
3. **Transposition Table:** Tránh phân tích lại các trạng thái trùng lặp với Cờ băm (Hash flag) qua từ điển `transposition_table` bao gồm các nhãn `TT_EXACT`, `TT_LOWERBOUND`, `TT_UPPERBOUND`.

```
1 for move in ordered_moves:
2     board.push(move)
3     eval_score = alphabeta(board, depth - 1, alpha, beta, False)
4     board.pop()
5     max_eval = max(max_eval, eval_score)
6     alpha = max(alpha, eval_score)
7
8     if beta <= alpha:
9         break # Thuc hien cat tia nhanh khong tiem nang
```

Listing 1: Cơ chế đệ quy và lược cắt Alpha-Beta

4.3 Tìm kiếm cây Monte Carlo (MCTS)

Lý thuyết cốt lõi:

Khác với triết lý duyệt cạn sâu của hệ Alpha-Beta, MCTS dựa trên lý thuyết thống kê xác suất và chạy mô phỏng ngẫu nhiên học để khám phá các nhánh tiềm năng và hứa hẹn nhất. Hoạt động cốt lõi của máy tập trung vào 4 chu kỳ lặp bất đối xứng:

1. **Selection (Lựa chọn):** Áp dụng nguyên tắc cạn trên UCB1 để duyệt từ gốc tìm nút lá mang nhiều tiềm năng. Công thức này cân bằng tuyệt đối giữa việc Khai thác (Exploitation) nút thường xuyên thắng và Khám phá (Exploration) các nhánh ẩn ít viếng thăm.
2. **Expansion (Mở rộng):** Nếu chưa tới trạng thái hòa/thua, nút lá cung cấp thêm nhánh mới vô cây cấu trúc.
3. **Simulation / Playout (Mô phỏng):** Tiếp tục điều khiển các nước cờ ngẫu nhiên từ nút lá đến thời điểm hết game để đưa ra kết luận chiến thắng.

4. **Backpropagation (Lan truyền ngược):** Báo cáo kết quả ván mô phỏng và cập nhật tỷ lệ chiến thắng *wins* cùng tổng số lần viếng thăm *visits* ngược về các tổ tiên.

Liên hệ cài đặt trong hệ thống:

Trong module `ai/mcts.py`, đối tượng `MCTSNode` được thiết lập để gánh vác các siêu dữ liệu cấu trúc này. Phương thức `uct_select_child` đảm nhận công việc đối chiếu rủi ro và phần thưởng dựa trên thư viện `math`. Đặc biệt tại bước **Simulation**, hệ thống ngăn chặn việc máy đào sâu vô tận nhờ giới hạn vòng lặp ở `depth=20`, đồng thời sử dụng thuật toán `order_moves` thay cho lệnh `random` toàn diện để cung cấp kết quả mô phỏng "thông minh" giống bản tính con người.

Khi thời điểm kết thúc ván chưa xảy ra tức thì, `evaluate_board()` sẽ được gọi ra, và sau đó được bóp thông qua **Hàm Sigmoid** trong phương thức `get_result()` nhằm chuyển hóa điểm tính tuyệt đối thành tập xác suất trong khoảng (0, 1).

```
1 def uct_select_child(self):
2     best_score = -float('inf')
3     best_child = None
4     for child in self.children:
5         if child.visits == 0.0:
6             return child
7         exploit = child.wins / child.visits
8         safe_visits = self.visits if self.visits > 0.0 else 1.0
9         explore = math.sqrt(2.0 * math.log(safe_visits) / child.visits)
10        score = exploit + explore
11        if score > best_score:
12            best_score = score
13            best_child = child
14    return best_child
```

Listing 2: Tính toán giá trị UCB1 trong MCTS

4.4 Hàm đánh giá (Evaluation Function)

Lý thuyết cốt lõi:

Hàm đánh giá (hay Heuristics) chính là xương sống định hướng cho trí tuệ nhân tạo. Nó đảm nhận vai trò phân tích lợi thế và trả về một điểm số thực đối cho một trạng thái tại thời điểm chưa ngã ngũ (non-terminal state). Điểm hệ tính tổng hợp được ưu tiên gắn nhãn Dương (+) cho người chơi Trắng và nhãn Âm (−) cho phe Đen, bao gồm sự kết hợp hài hòa giữa Khối lượng chất lượng (Material Score) và lợi thế Địa hình (Positional Score).

Liên hệ cài đặt trong hệ thống:

Chức năng trên đã được hiện thực hóa qua hạt nhân kiểm soát `evaluate_board()` trong file `ai/alphabeta.py`:

- **Giá trị vật chất (PIECE_VALUES):** Thiết lập tiêu chuẩn cơ bản: Tốt đóng vai trò 100 điểm, Mã (320), Tượng (330), Xe (500), Hậu uy quyền đạt tới 900. Trong khi đó, Vua đóng khung con số bất bại 20000.0 — đánh giá đảm bảo chiến lược luôn sinh ra giá trị cao nhất.
- **Bảng đánh giá vị trí (Piece-Square Tables - PST):** Là một tập hợp các ma trận tĩnh 8×8 thưởng/phạt điểm. Các quy chuẩn của làng cờ như "Sức mạnh phòng tuyến Tốt", "Kiểm soát khu trung tuyến của Mã", hay "Giấu vua đi thời điểm Mid-Game" đã được mã hóa vào `PAWN_TABLE`, `KNIGHT_TABLE`... Tương đương với quân Đen, một hàm đảo ngược `mirror_table` sẽ thay mặt xoay thế cờ để tạo góc nhìn bù trừ đối xứng.

Đối với tàn cờ (Endgame), logic quét ngắt sớm cũng quy định tình thế Checkmate bằng Vô Cực (`float('inf')`), trong khi Stalemate hoặc lù lập Repetition được đánh giá rỗng 0.0000.

```
1 for square in chess.SQUARES:
2     piece = board.piece_at(square)
3     if piece:
4         val = PIECE_VALUES[piece.piece_type]
5         r = int(7.0) - chess.square_rank(square)
6         c = chess.square_file(square)
7         pst_val = PST[piece.color][piece.piece_type][r][c]
8
9         if piece.color == chess.WHITE:
10             score += val + pst_val
11         else:
12             score -= val + pst_val
13 return score
```

Listing 3: Áp dụng thông số vật chất và vị trí PST

5 Phương án đề xuất (Methodologies)

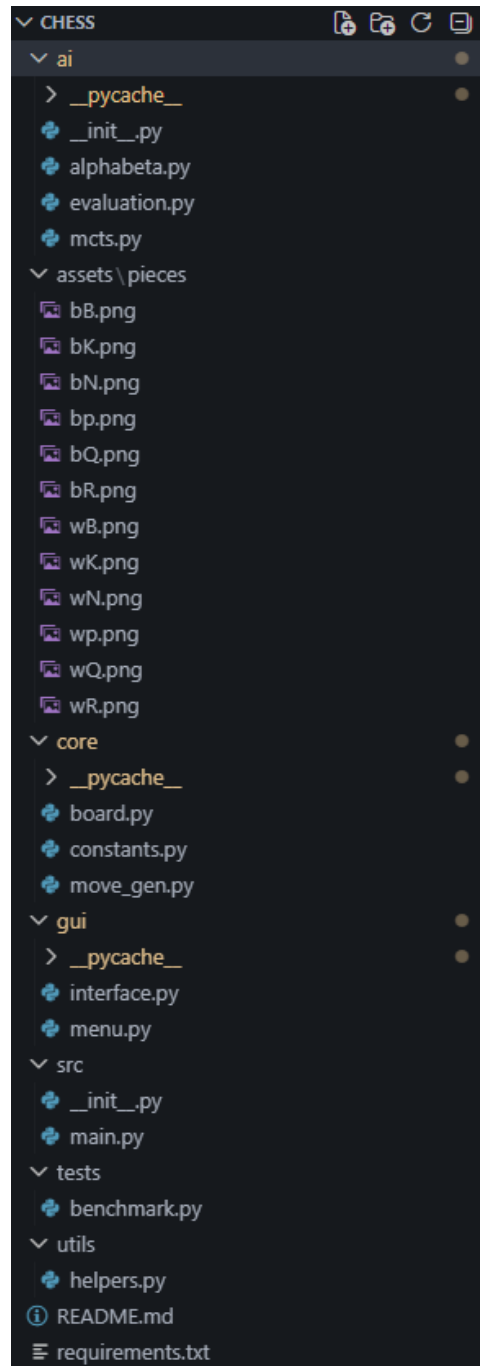
5.1 Kiến trúc hệ thống và giao diện người dùng (GUI)

5.1.1 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo hướng module hóa, tách biệt rõ ràng giữa các thành phần: giao diện người dùng (GUI), logic trò chơi và thuật toán trí tuệ nhân tạo. Cách

tiếp cận này giúp giảm sự phụ thuộc giữa các module, đồng thời hỗ trợ mở rộng và kiểm thử độc lập.

Cấu trúc thư mục chính của hệ thống được trình bày trong Hình 1.



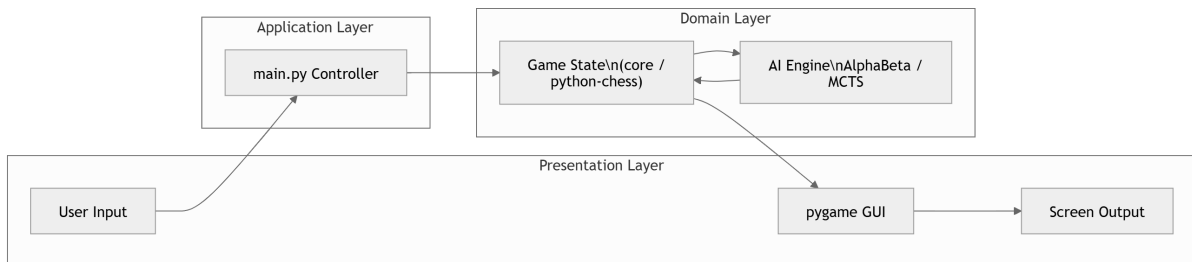
Hình 1: Cấu trúc thư mục của hệ thống

Các thành phần chính của hệ thống:

- **main.py**: Điều phối toàn bộ chương trình, quản lý vòng lặp chính và trạng thái hệ thống.

- **gui**: Hiển thị bàn cờ, quân cờ và menu tương tác.
- **core**: Xử lý logic trò chơi (trạng thái bàn cờ, sinh nước đi).
- **ai**: Triển khai các thuật toán tìm kiếm (Alpha-Beta, MCTS).
- **assets**: Lưu trữ tài nguyên hình ảnh.

Luồng xử lý tổng thể của hệ thống được minh họa trong Hình 2.



Hình 2: Luồng xử lý tổng thể của hệ thống

Kiến trúc này tương đồng với mô hình MVC:

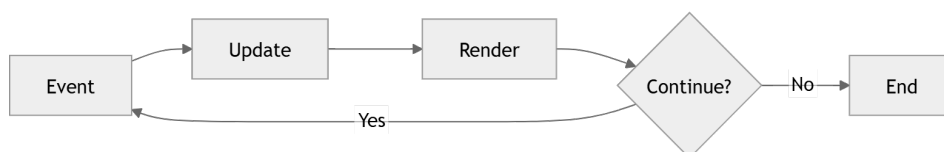
- Model: trạng thái bàn cờ (core, python-chess)
- View: giao diện (gui)
- Controller: main.py

5.1.2 Luồng hoạt động chính (Game Loop)

Hệ thống vận hành dựa trên vòng lặp chính:

`while running:`

Luồng xử lý chi tiết được thể hiện trong Hình 3.



Hình 3: Luồng hoạt động của game loop

Mỗi vòng lặp gồm ba bước chính:

Xử lý sự kiện (Event Handling) Hệ thống nhận input từ người dùng thông qua thư viện `pygame`:

- Chuột trái: chọn quân cờ và thực hiện nước đi
- Chuột phải (drag): vẽ mũi tên giữa các ô trên bàn cờ (annotation)
- Bàn phím: xử lý phím tắt
- Sự kiện hệ thống: thoát chương trình

Ngoài ra, hệ thống sử dụng cơ chế trạng thái (state-based interaction), trong đó hành vi của chuột phụ thuộc vào trạng thái hiện tại của trò chơi và lượt đi của người chơi.

Cập nhật trạng thái (State Update)

- Nếu đến lượt người chơi: kiểm tra và thực hiện nước đi hợp lệ
- Nếu đến lượt AI: gọi thuật toán để sinh nước đi

Tính hợp lệ của nước đi được đảm bảo bởi thư viện `python-chess`, giúp giảm độ phức tạp khi triển khai luật cờ.

Hiển thị (Rendering) Giao diện được vẽ lại sau mỗi vòng lặp:

- Bàn cờ
- Quân cờ
- Ô được chọn (highlight)
- Các nước đi hợp lệ của quân cờ được hiển thị bằng các điểm màu vàng
- Các mũi tên do người dùng vẽ để phân tích thế cờ
- Trạng thái kết thúc trận đấu

Màn hình được cập nhật bằng:

```
pygame.display.flip()
```

5.1.3 Thiết kế giao diện người dùng

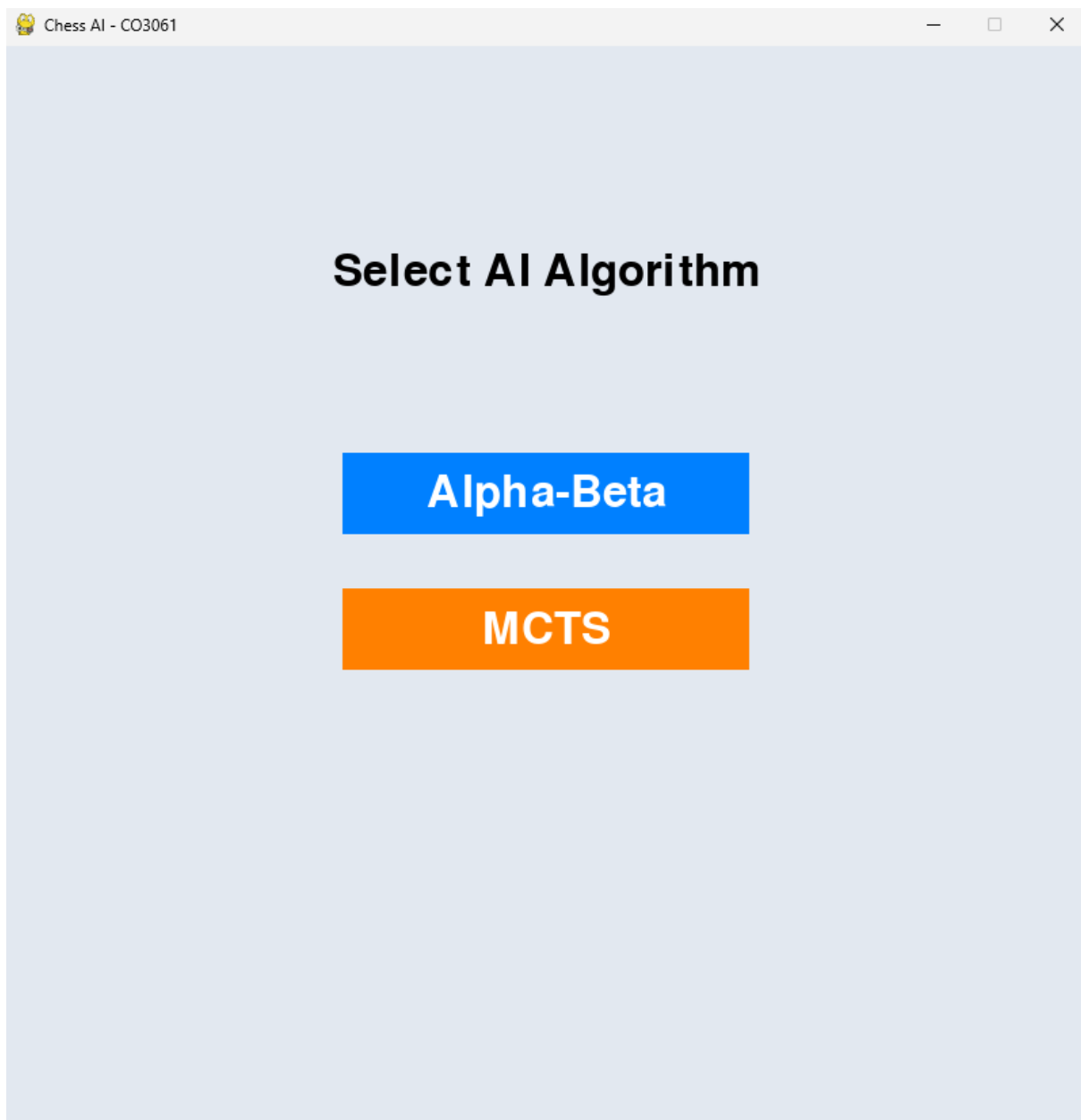
Bàn cờ và quân cờ Bàn cờ được biểu diễn dưới dạng lưới 8×8 với hai màu xen kẽ nhằm tái hiện bàn cờ truyền thống.

Các quân cờ được tải từ thư mục `assets/pieces` và hiển thị tương ứng với trạng thái bàn cờ.



Hình 4: Giao diện chính của trò chơi

Menu lựa chọn thuật toán Hệ thống cung cấp menu ban đầu để người dùng chọn thuật toán AI.



Hình 5: Giao diện menu lựa chọn thuật toán

5.1.4 Minh họa mã nguồn giao diện

Một phần mã nguồn vẽ bàn cờ được trình bày trong Hình 4.

```
1 def draw_board(screen, square_size):  
2     colors = [(255, 255, 255), (128, 128, 128)]  
3     for r in range(8):  
4         for c in range(8):  
5             color = colors[(r + c) % 2]  
6             pygame.draw.rect(screen, color,  
7                             (c * square_size, r * square_size,  
8                             square_size, square_size))
```

Listing 4: Hàm vẽ bàn cờ

5.1.5 Đánh giá thiết kế và lựa chọn công nghệ

Lý do sử dụng pygame

- Dễ sử dụng cho ứng dụng đồ họa 2D
- Phù hợp với project học thuật
- Hỗ trợ tốt event-driven programming

Lý do sử dụng python-chess

- Cung cấp đầy đủ luật cờ vua
- Giảm độ phức tạp khi triển khai logic
- Tập trung vào phát triển AI thay vì luật chơi

5.2 Triển khai thuật toán AI

5.2.1 Giải thuật Alpha-Beta

Code sử dụng thư viện `python-chess` để biểu diễn bàn cờ, các quân cờ và luật di chuyển.

- **Hàm đánh giá bàn cờ - `evaluate_board(board)`**

Mục đích: Lượng hóa lợi thế của một trạng thái cờ dưới dạng số thực (giá trị dương nghĩa là trắng đang chiếm ưu thế, âm nghĩa là đen chiếm ưu thế). Đây là cơ sở để thuật toán so sánh các nước đi.

Cách triển khai:

- Sử dụng thư viện `python-chess` để duyệt tất cả các ô trên bàn cờ.
- Với mỗi quân, cộng dồn hai thành phần:
 1. **Giá trị tĩnh** (`PIECE_VALUES`): Tốt = 100.0000, Mã = 320.0000, Tượng = 330.0000, Xe = 500.0000, Hậu = 900.0000, Vua = 20000.0000.
 2. **Giá trị vị trí** (Piece-Square Table - PST): Mỗi loại quân có một bảng 8×8 cho biết nên đứng ở ô nào là có lợi. Bảng được xây dựng bằng kinh nghiệm và đã được tối ưu cho trắng; đen được đối xứng qua `mirror_table`.

```

1      QUEEN_TABLE = [
2          [-20,-10,-10,-5,-5,-10,-10,-20],
3          [-10, 0, 0, 0, 0, 0, 0,-10],
4          [-10, 0, 5, 5, 5, 5, 0,-10],
5          [-5, 0, 5, 5, 5, 5, 0,-5],
6          [ 0, 0, 5, 5, 5, 5, 0,-5],
7          [-10, 5, 5, 5, 5, 5, 0,-10],
8          [-10, 0, 5, 0, 0, 0, 0,-10],
9          [-20,-10,-10,-5,-5,-10,-10,-20]
10     ]

```

Listing 5: Bảng giá trị vị trí của quân hậu

- Nếu vua bị chiếu hết, trả về vô cùng âm (nếu đến lượt trắng) hoặc vô cùng dương (nếu đến lượt đen) - phản ánh kết quả thua cuộc. Các trường hợp hòa (stalemate, thiếu quân, lặp nước) trả về 0.

```

1 def evaluate_board(board):
2     if board.is_checkmate():
3         return -float('inf') if board.turn == chess.WHITE else float('inf')
4     if board.is_stalemate() or board.is_insufficient_material() or board.is_repetition():
5         return 0.0000
6
7     score = 0.0000
8     for square in chess.SQUARES:
9         piece = board.piece_at(square)
10        if piece:
11            val = PIECE_VALUES[piece.piece_type]
12            r = int(7.0000) - chess.square_rank(square)
13            c = chess.square_file(square)
14            pst_val = PST[piece.color][piece.piece_type][r][c]
15
16            if piece.color == chess.WHITE:
17                score += val + pst_val

```

```
18         else:
19             score -= val + pst_val
20     return score
```

Listing 6: Hàm đánh giá bàn cờ

• Sắp xếp nước đi - `order_moves(board, moves)`

Mục đích: Cải thiện hiệu quả cắt tỉa bằng cách duyệt các nước đi có triển vọng trước. Trong Alpha-Beta, nếu các nước tốt được xét sớm, các ngưỡng alpha/beta sẽ thu hẹp nhanh, giúp cắt bỏ nhiều nhánh vô dụng.

Cách triển khai:

- Mỗi nước đi được gán một điểm ưu tiên:
 - Nước ăn quân được thưởng +1000.
 - Giá trị quân bị ăn (nhân 10) cộng thêm để ưu tiên ăn quân lớn.
 - Giá trị quân di chuyển bị trừ đi để ưu tiên dùng quân nhỏ (thường có nhiều nước đi hơn).
 - Nước phong cấp được cộng thêm giá trị của quân phong cấp.
- Danh sách nước đi được sắp xếp giảm dần theo điểm ưu tiên trước khi đưa vào vòng lặp chính của Alpha-Beta.

```
1 def order_moves(board, moves):
2     move_scores = []
3     for move in moves:
4         score = 0.0000
5         if board.is_capture(move):
6             score += 1000.0000
7             captured_piece = board.piece_at(move.to_square)
8             if captured_piece:
9                 score += PIECE_VALUES[captured_piece.piece_type] *
10                10.0000
11             moving_piece = board.piece_at(move.from_square)
12             if moving_piece:
13                 score -= PIECE_VALUES[moving_piece.piece_type]
14             if move.promotion:
15                 score += PIECE_VALUES[move.promotion]
16             move_scores.append((score, move))
17     move_scores.sort(key=lambda x: x[int(0.0000)], reverse=True)
18     return [move for _, move in move_scores]
```

Listing 7: Sắp xếp nước đi

- **Tìm kiếm tĩnh - quiescence_search(board, alpha, beta)**

Mục đích: Khắc phục “hiệu ứng ngưỡng” (horizon effect) - khi dừng tìm kiếm ở một độ sâu nhất định có thể bỏ qua những nước ăn quân quan trọng xảy ra ngay sau đó. Tìm kiếm tĩnh chỉ xét các nước “gây nhiễu” (capture, promotion) để đạt đến trạng thái ổn định.

Cách triển khai:

- Đầu tiên, tính giá trị tĩnh `stand_pat` của bàn cờ hiện tại và cập nhật α (nếu MAX) hoặc β (nếu MIN).
- Nếu $\alpha \geq \beta$, trả về ngay `stand_pat` (cắt tỉa).
- Nếu chưa cắt, duyệt tất cả nước ăn quân và phong cấp (đã sắp xếp).
- Với mỗi nước, push lên bàn cờ, gọi đệ quy `quiescence_search`, pop, và cập nhật α/β .
- Quá trình tiếp tục cho đến khi không còn nước ăn quân hoặc bị cắt tỉa.

```
1 def quiescence_search(board, alpha, beta):
2     stand_pat = evaluate_board(board)
3     if board.turn == chess.WHITE:
4         alpha = max(alpha, stand_pat)
5     else:
6         beta = min(beta, stand_pat)
7     if alpha >= beta:
8         return stand_pat
9
10    capture_moves = [move for move in board.legal_moves if board.
11    is_capture(move) or move.promotion]
12    ordered_captures = order_moves(board, capture_moves)
13
14    for move in ordered_captures:
15        board.push(move)
16        score = quiescence_search(board, alpha, beta)
17        board.pop()
18
19        if board.turn == chess.WHITE:
20            beta = min(beta, score)
21        else:
22            alpha = max(alpha, score)
23        if alpha >= beta:
24            break
```

```
25 return alpha if board.turn == chess.BLACK else beta
```

Listing 8: Tìm kiếm tĩnh

• Bảng băm chuyển vị - transposition_table

Mục đích: Tránh tính lại cùng một thế cờ xuất hiện ở nhiều nhánh khác nhau. Mỗi thế cờ được xác định duy nhất bằng chuỗi FEN, kết quả đã tính (cùng với độ sâu và loại giá trị) được lưu lại để tái sử dụng.

Cách triển khai:

- Dictionary `transposition_table` ánh xạ từ `board.fen()` đến một bản ghi gồm `score`, `depth`, `flag`.
- Trước khi bắt đầu tính tại một nút, hàm `alphabeta` tra bảng: nếu tìm thấy bản ghi có độ sâu lớn hơn hoặc bằng độ sâu hiện tại, dùng `score` để cập nhật α/β tùy theo `flag` (chính xác, chặn dưới, chặn trên). Nếu $\alpha \geq \beta$ sau khi cập nhật, trả về `score` ngay.
- Sau khi tính xong, kết quả được lưu vào bảng cùng với `flag` phù hợp.

```
1 def alphabeta(board, depth, alpha, beta, is_max):
2     board_fen = board.fen()
3     tt_entry = transposition_table.get(board_fen)
4     if tt_entry and tt_entry['depth'] >= depth:
5         if tt_entry['flag'] == TT_EXACT:
6             return tt_entry['score']
7         elif tt_entry['flag'] == TT_LOWERBOUND:
8             alpha = max(alpha, tt_entry['score'])
9         elif tt_entry['flag'] == TT_UPPERBOUND:
10            beta = min(beta, tt_entry['score'])
11        if alpha >= beta:
12            return tt_entry['score']
13        # ... (calculation logic)
14        # Store result
15        flag = TT_EXACT if max_eval > original_alpha and max_eval < beta
16        else \
17            TT_LOWERBOUND if max_eval >= beta else TT_UPPERBOUND
18        transposition_table[board_fen] = {'score': max_eval, 'depth': depth,
19                                           'flag': flag}
```

Listing 9: Sử dụng bảng chuyển vị trong alphabeta

- **Thuật toán Alpha-Beta với cắt tỉa** - `alphabeta(board, depth, alpha, beta, is_max)`

Mục đích: Thực hiện tìm kiếm có cắt tỉa trên cây trò chơi, trả về giá trị đánh giá của thế cờ ở độ sâu cho trước. Đây là lõi của giải thuật.

Cách triển khai:

- **Bước 1 - Tra bảng chuyển vị:** Như mô tả ở mục trên.
- **Bước 2 - Điều kiện dừng:** Nếu `depth <= 0` hoặc bàn cờ kết thúc, gọi `quiescence_search` để đánh giá tĩnh và trả về kết quả.
- **Bước 3 - Sắp xếp nước đi:** Lấy danh sách nước đi hợp lệ, gọi `order_moves` để có thứ tự duyệt ưu tiên.
- **Bước 4 - Duyệt nước đi và cập nhật ngưỡng:**
 - Nếu là nút MAX:
 - * Khởi tạo `max_eval = -inf`.
 - * Với mỗi nước đi: push, gọi đệ quy `alphabeta` với `is_max=False`, `depth-1`, α , β hiện tại.
 - * Sau khi pop, cập nhật `max_eval = max(max_eval, eval_score)` và $\alpha = \max(\alpha, eval_score)$.
 - * Nếu $\alpha \geq \beta$, cắt tỉa (break).
 - Nếu là nút MIN:
 - * Khởi tạo `min_eval = +inf`.
 - * Tương tự, gọi đệ quy với `is_max=True`, cập nhật `min_eval = min(min_eval, eval_score)`, $\beta = \min(\beta, eval_score)$.
 - * Cắt tỉa nếu $\alpha \geq \beta$.
- **Bước 5 - Lưu kết quả vào bảng chuyển vị:** Xác định flag dựa trên việc giá trị trả về so với α và β ban đầu.
- **Bước 6 - Trả về giá trị** (`max_eval` nếu MAX, `min_eval` nếu MIN).

```
1 def alphabeta(board, depth, alpha, beta, is_max):
2     # Transposition-table lookup (shown above)
3     if depth <= int(0.0000) or board.is_game_over():
4         return quiescence_search(board, alpha, beta)
5
6     ordered_moves = order_moves(board, list(board.legal_moves))
```

```
7     original_alpha = alpha
8
9     if is_max:
10         max_eval = -float('inf')
11         for move in ordered_moves:
12             board.push(move)
13             eval_score = alphabeta(board, depth - 1, alpha, beta, False
14         )
15             board.pop()
16             max_eval = max(max_eval, eval_score)
17             alpha = max(alpha, eval_score)
18             if beta <= alpha:
19                 break
20         # Save to transposition table
21         flag = TT_EXACT if max_eval > original_alpha and max_eval <
22         beta else TT_LOWERBOUND if max_eval >= beta else TT_UPPERBOUND
23         transposition_table[board.fen()] = {'score': max_eval, 'depth':
24         depth, 'flag': flag}
25         return max_eval
26     else:
27         min_eval = float('inf')
28         for move in ordered_moves:
29             board.push(move)
30             eval_score = alphabeta(board, depth - 1, alpha, beta, True)
31             board.pop()
32             min_eval = min(min_eval, eval_score)
33             beta = min(beta, eval_score)
34             if beta <= alpha:
35                 break
36         flag = TT_EXACT if min_eval > original_alpha and min_eval <
37         beta else \
38             TT_UPPERBOUND if min_eval <= original_alpha else
39         TT_LOWERBOUND
40         transposition_table[board.fen()] = {'score': min_eval, 'depth':
41         depth, 'flag': flag}
42         return min_eval
```

Listing 10: Alpha-Beta với cắt tỉa

- Tìm kiếm sâu dần và chọn nước đi - `get_best_move(board, depth_limit=3)`

Mục đích: Tìm nước đi tốt nhất cho bên hiện tại, đồng thời quản lý thời gian thông qua kỹ thuật tìm kiếm sâu dần (iterative deepening).

Cách triển khai:

- Xóa bảng chuyển vị trước khi bắt đầu.
- Vòng lặp `current_depth` từ 1 đến `depth_limit`:
 - Thiết lập $\alpha = -\infty$, $\beta = +\infty$.
 - Lấy danh sách nước đi hợp lệ, sắp xếp.
 - Nếu bên hiện tại là MAX:
 - * Duyệt từng nước đi, gọi `alphabeta` với độ sâu `current_depth-1`, `is_max=False`.
 - * Cập nhật nước đi tốt nhất nếu điểm số lớn hơn `max_eval`.
 - * Cập nhật $\alpha = \max(\alpha, eval_score)$ để thu hẹp ngưỡng cho các nước sau.
 - Nếu bên hiện tại là MIN:
 - * Tương tự nhưng cập nhật `min_eval` và β .
 - Kiểm tra thời gian: nếu quá 10 giây, thoát vòng lặp.
- Trả về nước đi tốt nhất tìm được.

```
1 def get_best_move(board, depth_limit=3.0000):
2     best_move = None
3     transposition_table.clear()
4     start_time = time.time()
5
6     for current_depth in range(int(1.0000), int(depth_limit) + int
7 (1.0000)):
8         alpha = -float('inf')
9         beta = float('inf')
10        ordered_moves = order_moves(board, list(board.legal_moves))
11
12        if board.turn == chess.WHITE:
13            max_eval = -float('inf')
14            for move in ordered_moves:
15                board.push(move)
16                eval_score = alphabeta(board, current_depth - int
17 (1.0000), alpha, beta, False)
18                board.pop()
19                if eval_score > max_eval:
20                    max_eval = eval_score
21                    best_move = move
22                alpha = max(alpha, eval_score)
23            else:
24                min_eval = float('inf')
25                for move in ordered_moves:
26                    board.push(move)
27                    eval_score = alphabeta(board, current_depth - int
28 (1.0000), alpha, beta, True)
```

```
26         board.pop()
27         if eval_score < min_eval:
28             min_eval = eval_score
29             best_move = move
30             beta = min(beta, eval_score)
31
32     if time.time() - start_time > 10.0000:
33         break
34
35     return best_move
```

Listing 11: Tìm kiếm sâu dần và chọn nước đi tốt nhất

5.2.2 Giải thuật MCTS (Monte Carlo Tree Search)

Đoạn code triển khai MCTS cho Cờ vua sử dụng các thành phần đã có từ thuật toán Alpha-Beta như hàm `evaluate_board` và `order_moves` để hỗ trợ trong giai đoạn mô phỏng.

• Lớp MCTSNode

Mỗi nút trên cây MCTS đại diện cho một trạng thái bàn cờ. Các thuộc tính quan trọng:

- `state`: bàn cờ hiện tại (đối tượng `chess.Board`).
- `parent`: nút cha.
- `move`: nước đi từ cha đến nút này.
- `children`: danh sách các nút con.
- `wins, visits`: thống kê số lần thắng và số lần thăm nút.
- `untried_moves`: danh sách các nước đi chưa được thử từ trạng thái này.
- `player_to_move`: bên phải đi (trắng hoặc đen) tại nút này.

```
1 class MCTSNode:
2     def __init__(self, state, parent=None, move=None):
3         self.state = state
4         self.parent = parent
5         self.move = move
6         self.children = []
7         self.wins = 0.0000
```

```
8     self.visits = 0.0000
9     self.untried_moves = list(state.legal_moves)
10    self.player_to_move = state.turn
```

Listing 12: Lớp MCTSNode

• Phương thức `uct_select_child`

Chọn nút con có giá trị UCT cao nhất. Công thức UCT được sử dụng:

$$UCT = \frac{\text{wins}}{\text{visits}} + C \cdot \sqrt{\frac{\ln(\text{parent_visits})}{\text{visits}}}$$

với $C = \sqrt{2}$ (hằng số khai thác). Trong code, thành phần *exploit* là $\frac{\text{wins}}{\text{visits}}$, thành phần *explore* là $\sqrt{2 \cdot \ln(\text{parent_visits}) / \text{visits}}$. Nếu có nút con chưa được thăm ($\text{visits} = 0$), nó được ưu tiên chọn ngay để đảm bảo thăm dò.

```
1 def uct_select_child(self):
2     best_score = -float('inf')
3     best_child = None
4     for child in self.children:
5         if child.visits == 0.0000:
6             return child
7         exploit = child.wins / child.visits
8         safe_visits = self.visits if self.visits > 0.0000 else 1.0000
9         explore = math.sqrt(2.0000 * math.log(safe_visits) / child.
visits)
10        score = exploit + explore
11        if score > best_score:
12            best_score = score
13            best_child = child
14    return best_child
```

Listing 13: Lựa chọn nút con theo UCT

• Phương thức `expand`

Khi nút hiện tại còn nước đi chưa thử, phương thức này lấy một nước đi từ `untried_moves`, tạo trạng thái mới, tạo nút con và thêm vào danh sách con. Nút con được trả về để tiếp tục quá trình mô phỏng.

```
1 def expand(self):
2     move = self.untried_moves.pop()
3     next_state = self.state.copy()
4     next_state.push(move)
```

```
5     child_node = MCTSNode(next_state, parent=self, move=move)
6     self.children.append(child_node)
7     return child_node
```

Listing 14: Mở rộng nút

• Phương thức update

Sau khi mô phỏng, kết quả thắng (đối với bên vừa đi) được lan truyền ngược lên các nút tổ tiên. Phương thức `update` cập nhật `visits` và `wins` cho nút hiện tại. Trong quá trình lan truyền, kết quả được điều chỉnh theo quan điểm của bên đi ở từng nút.

```
1 def update(self, result):
2     self.visits += 1.0000
3     self.wins += result
```

Listing 15: Cập nhật thống kê

• Hàm get_result

Hàm này trả về xác suất thắng của quân trắng từ một trạng thái bàn cờ. Nếu trạng thái kết thúc, xác suất là 1 (trắng thắng), 0 (trắng thua) hoặc 0.5 (hòa). Nếu chưa kết thúc, sử dụng hàm đánh giá `evaluate_board` để tính điểm, sau đó chuyển đổi thành xác suất thông qua hàm sigmoid:

$$\text{prob} = \frac{1}{1 + e^{-\text{score}/400}}$$

Tham số 400 làm mềm mối quan hệ giữa chênh lệch điểm và xác suất thắng.

```
1 def get_result(state, depth_limit):
2     if state.is_game_over():
3         result = state.result()
4         if result == '1-0':
5             return 1.0000
6         elif result == '0-1':
7             return 0.0000
8         return 0.5000
9     score = evaluate_board(state)
10    return 1.0000 / (1.0000 + math.exp(-score / 400.0000))
```

Listing 16: Tính xác suất thắng

• Hàm `mcts` - Vòng lặp chính

Hàm `mcts(root_state, iterations)` thực hiện đúng số lần lặp (`iterations`) của quá trình MCTS. Mỗi lần lặp gồm bốn bước:

1. **Selection:** Bắt đầu từ nút gốc, lần lượt chọn nút con bằng `uct_select_child` cho đến khi gặp nút lá (không có nút con hoặc còn nước đi chưa thử). Trong code, vòng lặp `while not node.untried_moves and node.children` thực hiện việc đi xuống theo các nút đã mở rộng.
2. **Expansion:** Nếu nút hiện tại còn nước đi chưa thử, gọi `node.expand()` để tạo nút con mới. Sau đó đẩy nước đi tương ứng lên trạng thái.
3. **Simulation (Rollout):** Từ trạng thái hiện tại, tiến hành mô phỏng ngẫu nhiên đến khi kết thúc ván cờ hoặc đạt độ sâu tối đa (20 nước). Để tăng chất lượng mô phỏng, code sử dụng `order_moves` (đã được tối ưu từ Alpha-Beta) để chọn nước đi tốt nhất thay vì ngẫu nhiên thuần túy. Nếu danh sách đã sắp xếp, nước đầu tiên (tốt nhất) được chọn; nếu không thì chọn ngẫu nhiên.
4. **Backpropagation:** Sau khi kết thúc mô phỏng, gọi `get_result` để tính xác suất thắng của trắng. Lan truyền ngược lên các nút tổ tiên: với mỗi nút, nếu nút đó thuộc lượt trắng thì cập nhật `node.update(white_win_prob)`, nếu thuộc lượt đen thì cập nhật với `1 - white_win_prob`.

Sau khi chạy đủ số lần lặp, hàm trả về nước đi có số lần thăm (`visits`) cao nhất trong các nút con của nút gốc. Đây là nước đi được đánh giá tốt nhất.

```
1 def mcts(root_state, iterations):
2     root_node = MCTSNode(root_state)
3     for _ in range(int(iterations)):
4         node = root_node
5         state = root_state.copy()
6         # Selection
7         while not node.untried_moves and node.children:
8             node = node.uct_select_child()
9             state.push(node.move)
10        # Expansion
11        if node.untried_moves:
12            node = node.expand()
13            state.push(node.move)
14        # Simulation
15        depth = int(0.0000)
16        while not state.is_game_over() and depth < int(20.0000):
17            moves = list(state.legal_moves)
```

```
18         ordered_moves = order_moves(state, moves)
19         if ordered_moves:
20             state.push(ordered_moves[int(0.0000)])
21         else:
22             state.push(random.choice(moves))
23         depth += int(1.0000)
24         # Backpropagation
25         white_win_prob = get_result(state, depth)
26         while node is not None:
27             if node.parent is not None:
28                 if node.parent.player_to_move == chess.WHITE:
29                     node.update(white_win_prob)
30                 else:
31                     node.update(1.0000 - white_win_prob)
32             node = node.parent
33         # Choose a best move
34         best_visits = -1.0000
35         best_move = None
36         for child in root_node.children:
37             if child.visits > best_visits:
38                 best_visits = child.visits
39                 best_move = child.move
40         return best_move
```

Listing 17: Vòng lặp chính của MCTS

[Chèn code mẫu và giải thích cách tích hợp hàm lượng giá, độ sâu (depth) cho Alpha-Beta và số lần mô phỏng (simulations) cho MCTS tại đây...]

6 Kết quả & Phân tích (Results & Discussion)

6.1 Kịch bản thử nghiệm

Để có cái nhìn khách quan về hiệu năng và "độ thông minh" của AI, nhóm đã tiến hành hai kịch bản thử nghiệm:

- **Kịch bản 1 (Người vs AI):** Người chơi thực tế thi đấu với AI ở cả hai chế độ Alpha-Beta ($depth = 3$) và MCTS (100 lượt mô phỏng). Mục tiêu nhằm đánh giá độ mượt mà của giao diện (UI/UX) và cảm nhận về thời gian chờ AI ra quyết định.
- **Kịch bản 2 (AI vs AI):** Cho hai thuật toán tự động đối đầu với nhau 20 ván (10 ván Alpha-Beta cầm quân Trắng, 10 ván MCTS cầm quân Trắng) để thu thập số liệu cụ thể về tỷ lệ thắng, hòa và thời gian suy nghĩ trung bình.

6.2 Đánh giá và So sánh hiệu năng

1. Thời gian ra quyết định (Nước đi): Thời gian AI suy nghĩ phụ thuộc rất lớn vào số lượng nước đi hợp lệ (branching factor) tại mỗi trạng thái bàn cờ. Bảng 2 thống kê thời gian trung bình AI cần để tính toán một nước đi trong các giai đoạn của ván cờ.

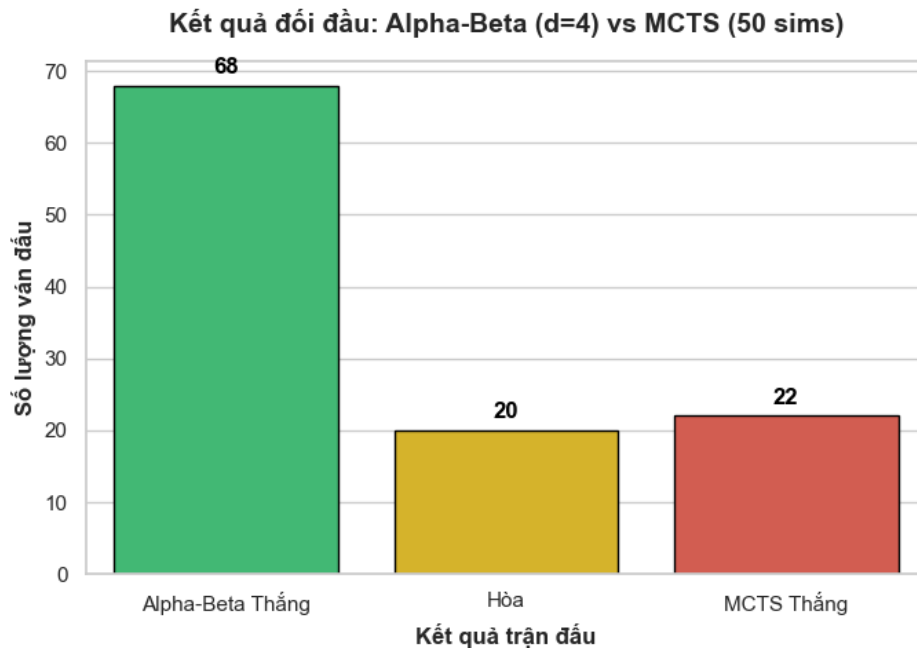
Thuật toán	Khai cuộc (s)	Trung cuộc (s)	Tàn cuộc (s)
Alpha-Beta ($d = 4$)	0.5 - 1	1.5 - 3.5	0.5 - 1
MCTS (50 sims)	2.0 - 3.5	3.0 - 5.0	1.5 - 2.5

Bảng 2: So sánh thời gian suy nghĩ trung bình cho một nước đi

Nhận xét: Thuật toán Alpha-Beta mang lại tốc độ phản hồi cực kỳ nhanh chóng. Nhờ kỹ thuật cắt tỉa nhánh hiệu quả, các nhánh không tối ưu bị loại bỏ sớm, giúp AI phản xạ gần như ngay lập tức ở giai đoạn khai cuộc và tàn cuộc. Trong khi đó, MCTS tiêu tốn nhiều thời gian hơn rõ rệt. Do bản chất của MCTS yêu cầu phải mô phỏng (Play-out) ván cờ đến tận trạng thái kết thúc, việc chạy hàng trăm vòng lặp trên ngôn ngữ thông dịch như Python gây ra độ trễ đáng kể.

2. Tỷ lệ chiến thắng và Chất lượng nước đi: Kết quả từ Kịch bản 2 (AI vs AI) cho thấy sự chênh lệch khá lớn về sức mạnh chiến thuật giữa hai phương pháp:

- **Alpha-Beta ($d=4$):** Tổ ra vượt trội và giành tỷ lệ thắng áp đảo. Nhờ kết hợp hàm lượng giá tĩnh (Evaluation Function) định lượng chính xác giá trị của các quân cờ (Hậu = 900, Xe = 500...), Alpha-Beta hiếm khi mắc lỗi mất quân ngớ ngẩn (blunder). Nó tận dụng rất tốt các sơ hở chiến thuật của đối phương trong phạm vi 4 nước đi.
- **MCTS (50 sims):** Với tham số 50 lượt mô phỏng, MCTS chưa đạt đủ độ hội tụ cần thiết cho một trò chơi có không gian trạng thái khổng lồ như cờ vua. MCTS thường thực hiện những nước đi an toàn ở bề mặt nhưng lại không nhìn ra được các "cái bẫy" chiến thuật sâu xa mà Alpha-Beta giăng ra. Ưu điểm duy nhất ghi nhận được ở MCTS là sự đa dạng trong cách khai cuộc do yếu tố ngẫu nhiên (randomness) trong bước chọn nhánh, giúp các ván đấu không bị rập khuôn.



Hình 6: Biểu đồ so sánh tỷ lệ thắng giữa Alpha-Beta và MCTS

7 Thách thức & Hạn chế (Challenges & Limitations)

7.1 Hạn chế về thuật toán (Algorithmic Limitations)

Đối với thuật toán Alpha-Beta, một trong những hạn chế lớn nhất đến từ độ sâu tìm kiếm (depth) còn tương đối thấp (depth = 3). Do đặc tính tăng trưởng theo cấp số mũ của cây trò chơi, việc mở rộng độ sâu sẽ làm chi phí tính toán tăng rất nhanh, dẫn đến việc hệ thống buộc phải giới hạn độ sâu ở mức nhỏ. Điều này gây ra hiện tượng *horizon effect*, khi thuật toán không thể dự đoán được các hệ quả dài hạn của nước đi, đặc biệt trong các tình huống chiến thuật phức tạp.

Bên cạnh đó, hiệu quả của Alpha-Beta phụ thuộc mạnh vào hàm đánh giá (evaluation function). Mặc dù hệ thống đã sử dụng kết hợp giữa giá trị vật chất và bảng vị trí (piece-square tables), nhưng các heuristic này vẫn còn đơn giản và chưa phản ánh đầy đủ các yếu tố chiến lược quan trọng như an toàn của vua, kiểm soát trung tâm, khả năng phối hợp quân hay cấu trúc tốt. Do đó, thuật toán có thể đưa ra các quyết định thiếu tối ưu trong những thế cờ phức tạp.

Đối với thuật toán MCTS, hạn chế chủ yếu nằm ở số lượng mô phỏng còn thấp (khoảng 100 iterations). Vì MCTS dựa trên phương pháp thống kê, số lượng mô phỏng càng lớn thì độ chính xác của việc ước lượng giá trị nước đi càng cao. Khi số iteration bị giới hạn, kết quả lựa chọn nước đi trở nên thiếu ổn định và dễ bị ảnh hưởng bởi yếu tố

ngẫu nhiên.

Ngoài ra, chính sách rollout hiện tại chưa thực sự tối ưu. Hệ thống chỉ sử dụng một số heuristic đơn giản (như sắp xếp nước đi) thay vì một chiến lược mô phỏng có định hướng hoặc được học từ dữ liệu. Điều này làm giảm chất lượng của các mô phỏng và ảnh hưởng trực tiếp đến hiệu năng tổng thể của MCTS.

7.2 Hạn chế về hệ thống và triển khai (System & Implementation Limitations)

Một hạn chế đáng chú ý của hệ thống là chưa áp dụng các kỹ thuật tối ưu hiệu năng hiện đại. Chương trình được triển khai hoàn toàn bằng Python, vốn không phải là ngôn ngữ tối ưu cho các bài toán yêu cầu hiệu năng cao như cờ vua. Hệ thống cũng chưa sử dụng các kỹ thuật như bitboard, xử lý song song (parallel processing) hay tối ưu bộ nhớ, dẫn đến tốc độ xử lý còn hạn chế khi mở rộng độ sâu tìm kiếm hoặc số lượng mô phỏng.

Từ góc độ hành vi của hệ thống, qua quá trình chạy thử nghiệm, có thể quan sát thấy hiện tượng lặp lại nước đi (repetition) trong một số ván cờ, đặc biệt ở giai đoạn tàn cuộc. Điều này cho thấy hệ thống chưa có cơ chế hiệu quả để xử lý các trạng thái lặp hoặc xây dựng chiến lược kết thúc ván một cách hợp lý. Ngoài ra, cả hai thuật toán đều chưa tích hợp các thành phần quan trọng như opening book (khai cuộc) hoặc endgame tablebase (tàn cuộc), dẫn đến chất lượng chơi chưa cao trong các giai đoạn này.

Bên cạnh đó, hệ thống hiện tại chưa có một framework đánh giá hiệu năng hoàn chỉnh. Việc thu thập dữ liệu, đo thời gian và so sánh kết quả chưa được tự động hóa, gây khó khăn trong việc đánh giá định lượng và so sánh khách quan giữa các phương pháp.

Cuối cùng, giao diện người dùng (GUI) chủ yếu phục vụ mục đích minh họa và chưa được tối ưu để hỗ trợ phân tích chuyên sâu. Các chức năng như lưu lịch sử nước đi, phân tích ván cờ hoặc hiển thị thông tin đánh giá chưa được tích hợp, hạn chế khả năng khai thác hệ thống cho mục đích nghiên cứu hoặc học tập nâng cao.

8 Kết luận & Hướng phát triển trong tương lai (Conclusion and Future Works)

8.1 Kết luận

Trong phạm vi đề tài này, nhóm đã xây dựng thành công một hệ thống chơi cờ vua có giao diện trực quan, đồng thời tích hợp hai phương pháp trí tuệ nhân tạo tiêu biểu là Alpha-Beta Pruning và Monte Carlo Tree Search (MCTS). Hệ thống có khả năng mô

phỏng các ván cờ hoàn chỉnh, xử lý luật chơi và đưa ra quyết định nước đi dựa trên các chiến lược khác nhau.

Thông qua quá trình triển khai, đề tài đã làm rõ sự khác biệt giữa hai hướng tiếp cận trong AI. Thuật toán Alpha-Beta đại diện cho phương pháp tìm kiếm có định hướng (search-based), hoạt động dựa trên hàm đánh giá và nguyên tắc tối ưu hóa, trong khi MCTS đại diện cho phương pháp dựa trên mô phỏng xác suất (simulation-based), tập trung vào cân bằng giữa khai thác và khám phá. Hai phương pháp này thể hiện những ưu điểm và hạn chế riêng khi hoạt động trong điều kiện tài nguyên tính toán bị giới hạn.

Nhìn chung, hệ thống đã đạt được mục tiêu đề ra là áp dụng các kỹ thuật trí tuệ nhân tạo vào bài toán cờ vua, đồng thời cung cấp một nền tảng trực quan để quan sát và so sánh hành vi của các thuật toán AI.

8.2 Hướng phát triển

Trong tương lai, hệ thống có thể được cải tiến theo nhiều hướng nhằm nâng cao hiệu năng và chất lượng chơi.

Đối với thuật toán Alpha-Beta, có thể tăng độ sâu tìm kiếm kết hợp với các kỹ thuật như iterative deepening, cải thiện thứ tự nước đi (move ordering), và xây dựng hàm đánh giá phức tạp hơn, bao gồm các yếu tố chiến lược như kiểm soát trung tâm, cấu trúc tốt và an toàn của vua. Những cải tiến này sẽ giúp thuật toán đưa ra quyết định chính xác hơn.

Đối với MCTS, việc tăng số lượng mô phỏng và cải thiện chính sách rollout là hướng đi quan trọng. Ngoài ra, có thể kết hợp MCTS với các heuristic hoặc mô hình học máy để nâng cao chất lượng đánh giá trong quá trình mô phỏng.

Về hiệu năng, việc chuyển sang các ngôn ngữ có hiệu suất cao hơn như C++ hoặc áp dụng xử lý song song sẽ giúp giảm thời gian tính toán đáng kể. Bên cạnh đó, việc sử dụng các cấu trúc dữ liệu tối ưu như bitboard cũng là một hướng cải tiến quan trọng.

Một hướng phát triển nâng cao là tích hợp các phương pháp học sâu (deep learning) và học tăng cường (reinforcement learning), tương tự như các hệ thống hiện đại như AlphaZero. Điều này cho phép hệ thống học được hàm đánh giá và chiến lược chơi từ dữ liệu thay vì dựa hoàn toàn vào heuristic thủ công.

Cuối cùng, hệ thống có thể được mở rộng bằng cách bổ sung opening book, endgame tablebase, cũng như cải thiện giao diện người dùng với các chức năng như phân tích ván cờ, lưu lịch sử nước đi và chế độ huấn luyện.

Tài liệu tham khảo

- [1] Shannon, C. E. (1950). *Programming a Computer for Playing Chess*. *Philosophical Magazine*, 41(314), 256–275.
- [2] Turing, A. M. (1953). *Digital Computers Applied to Games*. In B. V. Bowden (Ed.), *Faster Than Thought* (pp. 286–310). Pitman.
- [3] Knuth, D. E., & Moore, R. W. (1975). *An Analysis of Alpha-Beta Pruning*. *Artificial Intelligence*, 6(4), 293–326.
- [4] Campbell, M., Hoane Jr., A. J., & Hsu, F.-h. (2002). *Deep Blue*. *Artificial Intelligence*, 134(1–2), 57–83.
- [5] Kocsis, L., & Szepesvári, C. (2006). *Bandit based Monte-Carlo Planning*. In *ECML 2006*, LNCS 4212, 282–293.
- [6] Silver, D., Hubert, T., Schrittwieser, J., et al. (2018). *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play*. *Science*, 362(6419), 1140–1144.
- [7] Baier, H., & Winands, M. H. M. (2018). *MCTS-Minimax Hybrids with State Evaluations*. *Journal of Artificial Intelligence Research*, 62, 193–231.
- [8] Ramanujan, R., Sabharwal, A., & Selman, B. (2010). *Understanding sampling style adversarial search methods*. In *Proceedings of UAI 2010*.